



EdgeFleet.AI

Full-Stack Engineering Assessment

Assessment Duration: 4 Days

Build a modern AI-powered Finance and Expense Tracking Platform with scalable backend systems, intelligent insights, chatbot integration, and polished product experience.

Role: Full-Stack Engineer

Location: New Delhi, India

Type: Full-Time (On-site)

1 Assessment Overview

At EdgeFleet.AI, we build intelligent systems where scalable software, AI infrastructure, and real-time applications come together seamlessly.

This assessment is designed to evaluate your ability to:

- Build production-grade full-stack applications
- Design scalable backend architectures
- Develop polished frontend experiences
- Integrate AI-powered capabilities into products
- Think from a product and user perspective
- Write clean, maintainable, and modular code
- Make strong architectural and engineering decisions

The challenge intentionally mirrors real startup engineering conditions where ownership, scalability, speed, and product quality matter equally.

2 Project Challenge

2.1 Objective

Build a modern **AI-Powered Finance and Expense Tracking Platform** that helps users manage transactions, monitor spending patterns, and generate intelligent financial insights.

The platform must support:

- User authentication and account management
- Expense and income tracking
- Financial analytics dashboard
- Budget management workflows
- AI-generated financial summaries
- AI chatbot assistant for financial queries
- Responsive and scalable architecture

3 Core Product Requirements

3.1 1. Frontend Application

Build a modern frontend application using:

- React / Next.js (preferred)

- TypeScript
- Responsive UI architecture
- Reusable component design

Frontend Features:

- Authentication screens
- Dashboard with analytics widgets
- Expense management interface
- Income tracking workflows
- Budget planning UI
- Transaction history
- Charts and visualizations
- Search and filtering
- Dark/light theme support
- Proper loading and error states

3.2 2. Backend Services

Build scalable backend services with:

- REST APIs
- Authentication middleware
- Database integration
- Proper validation and error handling
- Logging and observability

Suggested Stack:

- Node.js / Express / NestJS
- OR Python FastAPI

Backend Requirements:

- JWT Authentication
- CRUD APIs
- Role-based access

- Input validation
- Rate limiting
- API documentation
- Secure API architecture

3.3 3. Database Design

Use any relational or NoSQL database.

Expected Deliverables:

- Proper schema design
- Relationships and indexing
- Migration scripts
- Query optimization

Supported databases include:

- PostgreSQL
- MySQL
- MongoDB

3.4 4. AI Integration Requirements

Your platform must include the following AI capabilities:

3.4.1 AI Financial Summary Generator

Generate intelligent summaries based on user transactions and spending behavior.

Example features:

- Monthly expense summaries
- Spending pattern analysis
- Budget warnings
- Smart financial insights
- Savings recommendations

Example:

“You spent 32% more on food delivery this month compared to last month. Your highest spending category was transportation. Reducing discretionary expenses by 15% could improve your monthly savings target.”

3.4.2 AI Chatbot Assistant

Integrate an AI chatbot capable of answering finance-related product queries.

Example queries:

- “How much did I spend on food this month?”
- “What was my highest expense category?”
- “Summarize my spending trends.”
- “How can I optimize my monthly budget?”

The chatbot may use:

- OpenAI APIs
- Local LLMs
- Retrieval-based systems
- Rule-based AI workflows

3.5 5. Product Quality Expectations

The application should demonstrate:

- Strong UX thinking
- Clean UI consistency
- Modular architecture
- Scalable engineering patterns
- Secure implementation
- Performance optimization
- Clear product workflows

4 Bonus Features (Nice to Have)

- Real-time notifications
- Advanced analytics
- Multi-user collaboration
- AI-powered categorization

5 Architecture Expectations

Candidates are expected to demonstrate understanding of:

- Clean architecture principles
- Separation of concerns
- Scalable system design
- API design standards
- State management
- Security best practices
- Database optimization
- Deployment readiness

6 Submission Requirements

6.1 Mandatory Deliverables

1. Public or private GitHub repository
2. Consolidated Technical Report

6.2 GitHub Repository Requirements

Your repository should include:

- Clear folder structure
- Meaningful commits
- Proper README
- Setup instructions
- Screenshots/GIFs
- Deployment instructions
- Environment configuration guide

6.3 Technical Report Requirements

The report should include:

- System architecture
- Database schema explanation
- API structure

- AI integration explanation
- Product decisions and tradeoffs
- Challenges faced
- Future scalability considerations

7 What We Value Most

We care deeply about engineers who:

- Think beyond just writing code
- Build systems with ownership
- Prioritize scalability and maintainability
- Understand product thinking
- Communicate clearly
- Learn quickly and adapt fast

You are not expected to build a perfect product.

We are more interested in:

- Engineering decisions
- Product quality
- Architecture thinking
- Problem-solving approach
- Technical depth
- Code clarity and maintainability

8 Submission Instructions

Submit the following:

- GitHub Repository Link
- Deployed Application Link (optional but recommended)
- Technical Report

We look forward to seeing how you build intelligent, scalable systems.